EECS 3311
Sami Ali
220024568
10/09/2026

Stage 1 Project Design Report

1.1 Project Description

This project is an AI powered open world investigation game where the player takes the role of an investigator and solves different crimes. Instead of following one fixed storyline, the game uses structured case templates together with AI to create new investigations with different suspects, motives, evidence, locations, and events. Players can explore the world, investigate locations, collect and review evidence, question NPCs, follow leads, and make decisions that can change how a case develops. The target users are players who enjoy mystery, crime solving, exploration, and interactive stories. The main goal is to give the player more freedom in how they approach an investigation while still keeping the game consistent and understandable.

Most mystery and detective games follow a fixed story, so once a case is solved there is nothing new to find on a second playthrough. Players are also limited to a short list of actions the developers wrote in advance, which makes it hard to try their own ideas. This project addresses both problems by generating a new case each time and letting the player type what they want to do in their own words.

An AI agent is useful because the player can describe actions and questions in natural language instead of being limited to a fixed list of choices. The agent can interpret these actions, generate cases and dialogue, connect information from the investigation, remember important events, and create consequences based on the player's decisions. The project will use an LLM to provide these AI capabilities. The AI will work together with a deterministic game system rather than controlling the entire game. The game system will manage important rules and game state, such as available locations, evidence, player information, and valid actions. The agent will use this information to reason about the player's requests and

interact with the game through defined components. This combination allows the game to have flexible AI generated content while keeping the investigation logically consistent.

The LLM used in this project is Claude, accessed through the Anthropic API. It is called only through the LLMClient interface, so the model can be swapped without changing the rest of the system. The AI takes part in the project as an agent: it interprets what the player types, uses stored history to remember earlier events, decides how an NPC should answer, and plans a case in several steps (generate, check, regenerate if needed).

The game has both a GUI and a CLI. In the GUI the player uses the map, conversation, case file and character panels. In the CLI the same actions are typed as commands. Both send their input to the same GameController, so every feature works the same way in either one.

GameGUI or GameCLI passes the input to GameController, which asks AgentController to interpret it with the LLM. The result is a GameCommand. The game system validates the command, executes it on Game, updates the state, and notifies its observers so the GUI and memory update. The AI never changes the game state directly. It only produces commands and text, and the game system decides what actually happens.

1.2 Feature Specification

F01 - Player Creation

The user enters an investigator name and selects basic character attributes through the GUI. The system validates the information and creates an investigator profile that is displayed in the character panel. This feature is deterministic because no AI is required to create the player. If information is missing or invalid, the system asks the user to correct it before continuing.

F02 - Dynamic Case Generation

The user selects New Investigation through the GUI and can provide optional preferences for the type of case. The agent uses a structured case template and generates the specific crime, suspects, motives, evidence, locations, and possible leads. The feature is hybrid because the game system provides the rules while the LLM creates the case details. The generated case is

validated before being added to the game, and inconsistent information is rejected or regenerated.

F03 - World and Location Exploration

The user explores the game world through the GUI map and selects locations to investigate. The system uses the player's current state to determine which locations can be accessed and what interactions are available, while the AI can generate descriptions and contextual events. The selected location and available information are displayed in the GUI. If a location is restricted or unavailable, the system prevents access and explains why. This feature is hybrid because the game system decides which locations can be accessed, while the AI can write the descriptions and contextual events.

F04 - Natural Language Actions

The user enters an action through the conversation interface, such as asking to search a room, follow a suspect, or examine an object. The agent interprets the request and identifies the intended game action, then sends it to the game system for validation. The system produces the result and updates the game state if the action is valid. If the request is unclear, the agent asks for clarification, and if it is impossible, the system rejects it without changing the game state. This feature is hybrid because the AI interprets the request, but the game system validates it and decides the result.

F05 - NPC Interaction and Dynamic Dialogue

The user selects a suspect, witness, or other NPC and communicates with them through the conversation interface. The agent uses the NPC's information, personality, relationships, and previous interactions to generate context-sensitive dialogue. The conversation can reveal information, create leads, or affect relationships. NPCs may refuse to answer, provide incomplete information, or give misleading responses depending on the situation. This feature is hybrid because the LLM writes the dialogue, while the game system controls which facts the NPC is allowed to reveal.

F06 - Evidence Collection and Management

The user investigates locations and objects through the GUI to discover evidence, which is then displayed in the case file. The system determines what evidence can actually be found based on the current game state, while the AI can provide descriptions and context for discovered evidence. The output is an updated evidence collection that can be reviewed later. Evidence may be missed if the user does not perform the correct investigation action. This feature is hybrid because the game system decides what evidence can be found, while the AI only provides the descriptions.

F07 - Investigation and Lead Management

The user can review suspects, clues, relationships, and possible leads through an investigation board or case file in the GUI. The system stores confirmed information from the investigation, while the agent can identify possible connections and suggest useful directions. The output is an updated set of leads and connections that can guide the investigation. Leads may become irrelevant or change when new evidence is discovered. This feature is hybrid because the system stores the confirmed information, while the agent only suggests possible connections.

F08 - Rule and Action Validation

When the user or AI requests an action, the game system checks it against the current game state and rules. This feature is accessed automatically through normal gameplay rather than through a separate GUI control. The system produces either a valid action and updated state or a rejection message. This feature is deterministic and prevents the AI from creating impossible changes to the game. Invalid actions are rejected without changing the game state.

F09 - Consequence Generation

After an important player decision, the game system and agent determine how the investigation should react. The system updates deterministic information such as relationships, available actions, and case state, while the LLM generates the narrative consequence shown through the GUI and event log. The result may change future events, NPC behaviour, or available leads. Generated consequences must remain consistent with

established facts, and invalid results are rejected or regenerated. This feature is hybrid because the game system applies the deterministic changes and the LLM only writes the narrative.

F10 - Story and Investigation Memory

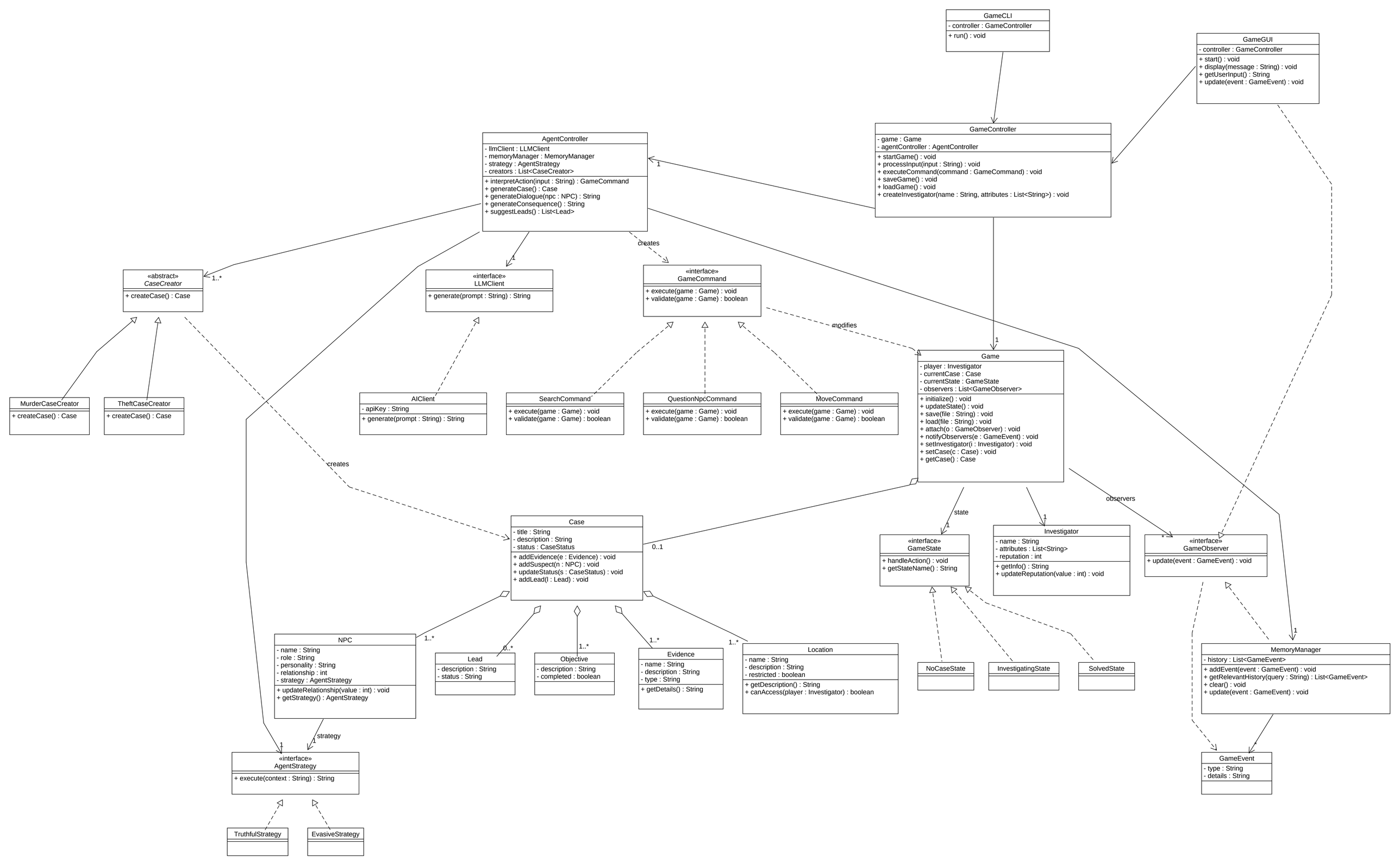
The user can review important previous events through the investigation history in the GUI, while the agent automatically retrieves relevant memories during later interactions. The system stores important conversations, evidence, decisions, and events, and the LLM uses relevant information when generating future responses. This allows the investigation to remain consistent across interactions. Irrelevant information can be excluded from the active context to prevent confusion. This feature is hybrid because the system stores and retrieves the events, while the LLM uses the relevant ones when it writes later responses.

F11 - Case Progress Tracking

The user can view current objectives and investigation progress through the case interface. The system updates objectives when required conditions are met, while the agent can generate new leads or investigative objectives as the case develops. The output is an updated list of completed and active objectives. Objectives may change or become unavailable when the player makes decisions that affect the investigation. This feature is hybrid because the system updates objectives from set conditions, while the agent can suggest new leads or objectives.

F12 - Save and Load Game

The user selects Save or Load through the GUI to preserve or continue an investigation. The system stores information such as the player's state, evidence, locations, NPC relationships, case progress, and relevant history, and restores it when a saved game is loaded. This feature is deterministic because it does not require the AI to save or restore the game state. Missing, invalid, or corrupted save data is rejected and the user is notified.


GameCLI
- controller : GameController
+ run() : void
GameGUI
- controller : GameController
+ start() : void
+ display(message : String) : void
+ getUserInput() : String
+ update(event : GameEvent) : void
GameController
- game : Game
- agentController : AgentController
+ startGame() : void
+ processInput(input : String) : void
+ executeCommand(command : GameCommand) : void
+ saveGame() : void
+ loadGame() : void
+ createInvestigator(name : String, attributes : List<String>) : void
AgentController
- llmClient : LLMClient
- memoryManager : MemoryManager
- strategy : AgentStrategy
- creators : List<CaseCreator>
+ interpretAction(input : String) : GameCommand
+ generateCase() : Case
+ generateDialogue(npc : NPC) : String
+ generateConsequence() : String
+ suggestLeads() : List<Lead>
«abstract»
CaseCreator
+ createCase() : Case
«interface»
LLMClient
+ generate(prompt : String) : String
«interface»
GameCommand
+ execute(game : Game) : void
+ validate(game : Game) : boolean
MurderCaseCreator
+ createCase() : Case
TheftCaseCreator
+ createCase() : Case
AIClient
- apiKey : String
+ generate(prompt : String) : String
SearchCommand
+ execute(game : Game) : void
+ validate(game : Game) : boolean
QuestionNpcCommand
+ execute(game : Game) : void
+ validate(game : Game) : boolean
MoveCommand
+ execute(game : Game) : void
+ validate(game : Game) : boolean
Game
- player : Investigator
- currentCase : Case
- currentState : GameState
- observers : List<GameObserver>
+ initialize() : void
+ updateState() : void
+ save(file : String) : void
+ load(file : String) : void
+ attach(o : GameObserver) : void
+ notifyObservers(e : GameEvent) : void
+ setInvestigator(i : Investigator) : void
+ setCase(c : Case) : void
+ getCase() : Case
Case
- title : String
- description : String
- status : CaseStatus
+ addEvidence(e : Evidence) : void
+ addSuspect(n : NPC) : void
+ updateStatus(s : CaseStatus) : void
+ addLead(l : Lead) : void
«interface»
GameState
+ handleAction() : void
+ getStateName() : String
Investigator
- name : String
- attributes : List<String>
- reputation : int
+ getInfo() : String
+ updateReputation(value : int) : void
«interface»
GameObserver
+ update(event : GameEvent) : void
NPC
- name : String
- role : String
- personality : String
- relationship : int
- strategy : AgentStrategy
+ updateRelationship(value : int) : void
+ getStrategy() : AgentStrategy
Lead
- description : String
- status : String
Objective
- description : String
- completed : boolean
Evidence
- name : String
- description : String
- type : String
+ getDetails() : String
Location
- name : String
- description : String
- restricted : boolean
+ getDescription() : String
+ canAccess(player : Investigator) : boolean
NoCaseState
InvestigatingState
SolvedState
MemoryManager
- history : List<GameEvent>
+ addEvent(event : GameEvent) : void
+ getRelevantHistory(query : String) : List<GameEvent>
+ clear() : void
+ update(event : GameEvent) : void
GameEvent
- type : String
- details : String
«interface»
AgentStrategy
+ execute(context : String) : String
TruthfulStrategy
EvasiveStrategy
creates
1
1..*
modifies
observers
state
0..1
0..*
strategy

2.1 Class Diagram

The class diagram has three main parts. The interface layer is GameGUI and GameCLI, which both use GameController. The AI and agent layer is AgentController, which uses the LLMClient interface (implemented by AIClient), MemoryManager and an AgentStrategy. The game system is Game together with Investigator, Case, NPC, Evidence, Location, Lead and Objective, plus the GameState, GameCommand and CaseCreator hierarchies. A Case is made up of its evidence, locations, NPCs, objectives and leads, and Game keeps a list of GameObserver objects.

Design Patterns:

The design uses five patterns. Each one solves a specific problem in the game.

Command

Problem: The LLM returns free text, but the game should only act on a fixed set of valid actions, and each action should be checked before it changes anything.

Classes and roles: GameCommand is the interface with validate() and execute(). MoveCommand, SearchCommand and QuestionNpcCommand are the concrete commands. AgentController creates the command from the player's text, GameController is the invoker (executeCommand()), and Game is the receiver.

Why it fits: Every request the AI interprets becomes a command object that is validated first and only executed if it is valid. This is how the AI is kept from changing the game directly.

Without it: GameController would need separate if/else code for every action, validation would be mixed into the execution code, and adding an action would mean editing existing code.

State

Problem: What the player is allowed to do depends on the phase of the game: no case yet, investigating, or solved.

Classes and roles: GameState is the interface (handleAction(), getStateName()). NoCaseState, InvestigatingState and SolvedState are the concrete states. Game is the context and holds currentState, which it changes in updateState().

Why it fits: Each state decides what is allowed in its phase, and Game only switches states when a case is created or solved.

Without it: Game would be full of repeated checks like "is there a case and is it unsolved?" in every method.

Strategy

Problem: NPCs should behave differently (answer honestly, dodge questions), and the dialogue code should not depend on which behaviour an NPC has.

Classes and roles: AgentStrategy is the interface with execute(context). TruthfulStrategy and EvasiveStrategy are the concrete strategies. NPC holds its strategy (getStrategy()) and AgentController uses it when it calls generateDialogue().

Why it fits: The behaviour is chosen per NPC and can change while the game runs, without changing AgentController.

Without it: generateDialogue() would have a large if/else on the NPC type, and every new behaviour would mean editing it.

Factory Method

Problem: Different crime types need different case setup, but the rest of the game only needs a Case.

Classes and roles: CaseCreator is the abstract creator with createCase(). MurderCaseCreator and TheftCaseCreator are the concrete creators. Case is the product. AgentController is the client and keeps a list of creators.

Why it fits: AgentController asks any CaseCreator for a Case without knowing the crime type. A new case type is just a new subclass.

Without it: generateCase() would contain type-specific code for every crime and would grow each time a type is added.

Observer

Problem: Several parts of the system (the GUI and the memory) must react whenever the game changes, but Game should not depend on them.

Classes and roles: Game is the subject (attach(), notifyObservers()). GameObserver is the interface with update(event). GameGUI and MemoryManager are the observers. GameEvent is the data that is sent.

Why it fits: The GUI and the memory stay in sync automatically, and new observers can be added without touching Game.

Without it: Game would call GameGUI and MemoryManager directly, so the classes would be tightly coupled and every new listener would mean editing Game.

AI Investigation Game (GUI and CLI)

UC01
Create Investigator
(F01)

UC02
Start New Investigation
(F02, F11)

UC03
Explore Location
(F03, F08)

UC08
Validate Action
(F08)

«include»

«include»

UC04
Perform Natural Language Action
(F04, F08)

«include»

«extend»

UC06
Collect Evidence
(F06)

«extend»

UC05
Interview NPC
(F05, F10)

UC09
Apply Consequences
(F09, F10)

UC07
Review Case File and Leads
(F07)

UC10
Review Investigation History
(F10)

UC11
Track Case Progress
(F11)

UC12
Save Game
(F12)

UC13
Load Game
(F12)

Player

LLM Service (AIClient)

File System

2.2 Use-Case Diagram and Use-Case Descriptions

Each use case below lists the sequence diagram that realizes it. In the GUI the player uses the panels. In the CLI the same actions are available through commands that call the same GameController methods.

UC01 - Create Investigator

Actor(s)	Player
Goal	Create a valid investigator profile.
Preconditions	Game is running and no investigator exists.
Trigger	Player clicks Create Investigator in the GUI (or uses the CLI command).
Main Success Scenario	1. Player enters a name and chooses attributes. 2. GameController.createInvestigator() checks the input. 3. An Investigator is created and passed to Game.setInvestigator(). 4. Game.notifyObservers() sends playerCreated and GameGUI.update() runs. 5. GameGUI shows the profile from Investigator.getInfo().
Alternative / Exception Flows	2a. Name is empty or attributes are invalid: error is shown, nothing is created, player corrects the input.
Postconditions	An Investigator is stored in Game.
Related Feature(s)	F01
Sequence Diagram	SD01

UC02 - Start New Investigation

Actor(s)	Player, LLM Service
Goal	Generate and start a new validated case.
Preconditions	An investigator exists and no case is active.
Trigger	Player clicks New Investigation.
Main Success Scenario	1. Player optionally picks a case type. 2. GameController.startGame() calls AgentController.generateCase(). 3. AgentController picks a CaseCreator (for example MurderCaseCreator) and calls createCase(). 4. The creator asks the LLMClient for case details and checks that suspects, locations, evidence and objectives exist. 5. GameController calls Game.setCase() and Game.updateState(); NoCaseState moves to InvestigatingState. 6. GameGUI shows the case briefing and objectives.
Alternative / Exception Flows	4a. Case is incomplete or inconsistent: it is regenerated, up to 3 attempts. 4b. LLM error or 3 failed attempts: error is shown and no case is added.
Postconditions	A valid Case is in Game and the state is InvestigatingState.
Related Feature(s)	F02, F11
Sequence Diagram	SD01

UC03 - Explore Location

Actor(s)	Player
Goal	Move to a location and see what can be done there.
Preconditions	A case is active (InvestigatingState).
Trigger	Player clicks a location on the map (or uses the go command).
Main Success Scenario	1. Player selects a location. 2. A MoveCommand is created and executed through GameController.executeCommand(). 3. UC08 Validate Action runs (Location.canAccess()). 4. Game updates the current location and notifies observers. 5. GameGUI shows Location.getDescription() and the available interactions.
Alternative / Exception Flows	3a. Location is restricted: reason is shown and the location does not change.
Postconditions	Current location changes only if the move was valid.
Related Feature(s)	F03, F08
Sequence Diagram	SD02

UC04 - Perform Natural Language Action

Actor(s)	Player, LLM Service
Goal	Carry out an action written in plain language.
Preconditions	A case is active.
Trigger	Player types an action in the conversation panel (or uses the CLI).
Main Success Scenario	1. Player types, for example, "search the study desk". 2. GameController.processInput() calls AgentController.interpretAction(). 3. The agent reads relevant events from MemoryManager and calls LLMClient.generate(). 4. A GameCommand (for example SearchCommand) is returned. 5. UC08 Validate Action runs, then the command is executed. 6. Observers update the GUI and MemoryManager.
Alternative / Exception Flows	3a. LLM error: player is asked to try again, state unchanged. 3b. Action unclear: clarification question is shown, state unchanged. 5a. Command rejected: reason is shown, state unchanged.
Postconditions	Game state changes only if the command was valid.
Related Feature(s)	F04, F08
Sequence Diagram	SD02

UC05 - Interview NPC

Actor(s)	Player, LLM Service
Goal	Question an NPC and get a reply that fits their behaviour and the case facts.
Preconditions	A case is active and the NPC is at the current location.
Trigger	Player selects an NPC and types a question (or uses the CLI).

Main Success Scenario	1. Player selects the NPC and asks a question. 2. A QuestionNpcCommand is validated and executed (UC08). 3. AgentController.generateDialogue() gets the NPC's AgentStrategy with NPC.getStrategy(). 4. The strategy decides which facts may be revealed (execute()). 5. The agent uses MemoryManager and LLMClient to write the reply and checks it against known facts. 6. NPC.updateRelationship() is called and the reply is displayed.
Alternative / Exception Flows	2a. NPC not present or case not active: rejected with a reason. 5a. Reply contradicts facts: regenerated up to 3 times, then a short fallback line is used. Evasive strategy: NPC avoids the question.
Postconditions	Relationship may change and the conversation is stored as a GameEvent.
Related Feature(s)	F05, F10
Sequence Diagram	SD03

UC06 - Collect Evidence

Actor(s)	Player
Goal	Find evidence and add it to the case file.
Preconditions	A case is active and the player is at a location.
Trigger	Player searches or examines something (extends UC04).
Main Success Scenario	1. Player types or selects a search or examine action. 2. A SearchCommand is validated and executed. 3. The command finds the Evidence for the target and calls Case.addEvidence(). 4. Game.notifyObservers(evidenceFound) updates the case file.
Alternative / Exception Flows	2a. Wrong action or nothing to find: player is told and nothing is added. 2b. Evidence already found: player is told.
Postconditions	The evidence collection grows only if something new was found.
Related Feature(s)	F06
Sequence Diagram	SD02

UC07 - Review Case File and Leads

Actor(s)	Player, LLM Service
Goal	See suspects, evidence and leads and get suggested connections.
Preconditions	A case is active.
Trigger	Player opens the case file or clicks Suggest Connections.
Main Success Scenario	1. Player opens the case file. 2. GameController gets the Case with Game.getCase() and GameGUI displays it. 3. Player clicks Suggest Connections. 4. AgentController.suggestLeads() asks the LLMClient for possible leads. 5. Leads that only use known evidence are added with Case.addLead(). 6. GameGUI refreshes the lead board.

Alternative / Exception Flows	5a. Suggestion refers to unknown evidence or LLM fails: empty list, existing leads only.
Postconditions	Lead list is up to date.
Related Feature(s)	F07
Sequence Diagram	SD05

UC08 - Validate Action

Actor(s)	None (internal)
Goal	Make sure only legal actions change the game.
Preconditions	A GameCommand has been created.
Trigger	GameController.executeCommand() is called (included by UC03, UC04, UC05).
Main Success Scenario	1. GameController calls command.validate(game). 2. The command asks the current GameState.handleAction() if the action is allowed. 3. The command checks its own rules (for example Location.canAccess()). 4. Result is returned and the command is executed only if valid.
Alternative / Exception Flows	3a. Any check fails: false is returned, the reason is displayed and the state does not change.
Postconditions	State is unchanged if validation failed.
Related Feature(s)	F08
Sequence Diagram	SD02

UC09 - Apply Consequences

Actor(s)	LLM Service
Goal	React to an important decision with state changes and a story result.
Preconditions	A command was executed and counted as a major decision (extends UC04).
Trigger	Game.updateState() finishes after a major decision.
Main Success Scenario	1. Deterministic changes (relationships, case state) are applied first. 2. AgentController.generateConsequence() asks the LLMClient for a narrative. 3. Text that contradicts established facts is rejected. 4. GameGUI.display() shows the narrative and the event is stored by MemoryManager.
Alternative / Exception Flows	3a. Narrative contradicts facts: regenerated, then plain text is shown. 2a. LLM unavailable: plain text of the deterministic changes is shown.
Postconditions	State reflects the decision and the event log has a new entry.
Related Feature(s)	F09, F10
Sequence Diagram	SD02

UC10 - Review Investigation History

Actor(s)	Player
Goal	Look back at important past events.
Preconditions	A case is active.
Trigger	Player opens the investigation history (or uses the history command).
Main Success Scenario	1. Player opens the history. 2. GameController calls MemoryManager.getRelevantHistory(query). 3. GameGUI displays the GameEvents in time order.
Alternative / Exception Flows	2a. No events yet: an empty history message is shown.
Postconditions	No state change.
Related Feature(s)	F10
Sequence Diagram	SD05

UC11 - Track Case Progress

Actor(s)	Player
Goal	See completed and active objectives.
Preconditions	A case is active.
Trigger	Player opens objectives or Game.updateState() runs after an action.
Main Success Scenario	1. Game.updateState() compares the state to each Objective. 2. Completed objectives are marked. 3. GameGUI displays completed and active objectives.
Alternative / Exception Flows	1a. A decision made an objective impossible: it is marked unavailable. 2a. Case is solved: state changes to SolvedState.
Postconditions	Objective list matches the game state.
Related Feature(s)	F11
Sequence Diagram	SD05, SD02

UC12 - Save Game

Actor(s)	Player, File System
Goal	Store the current investigation.
Preconditions	A case is active.
Trigger	Player clicks Save (or uses the save command).
Main Success Scenario	1. Player clicks Save. 2. GameController.saveGame() calls Game.save(file). 3. Game writes the player, case, evidence, relationships, progress and history to the file system. 4. GameGUI shows a confirmation.
Alternative / Exception Flows	3a. Write fails: error is shown and the previous save is kept.
Postconditions	A save file exists.
Related Feature(s)	F12
Sequence Diagram	SD04

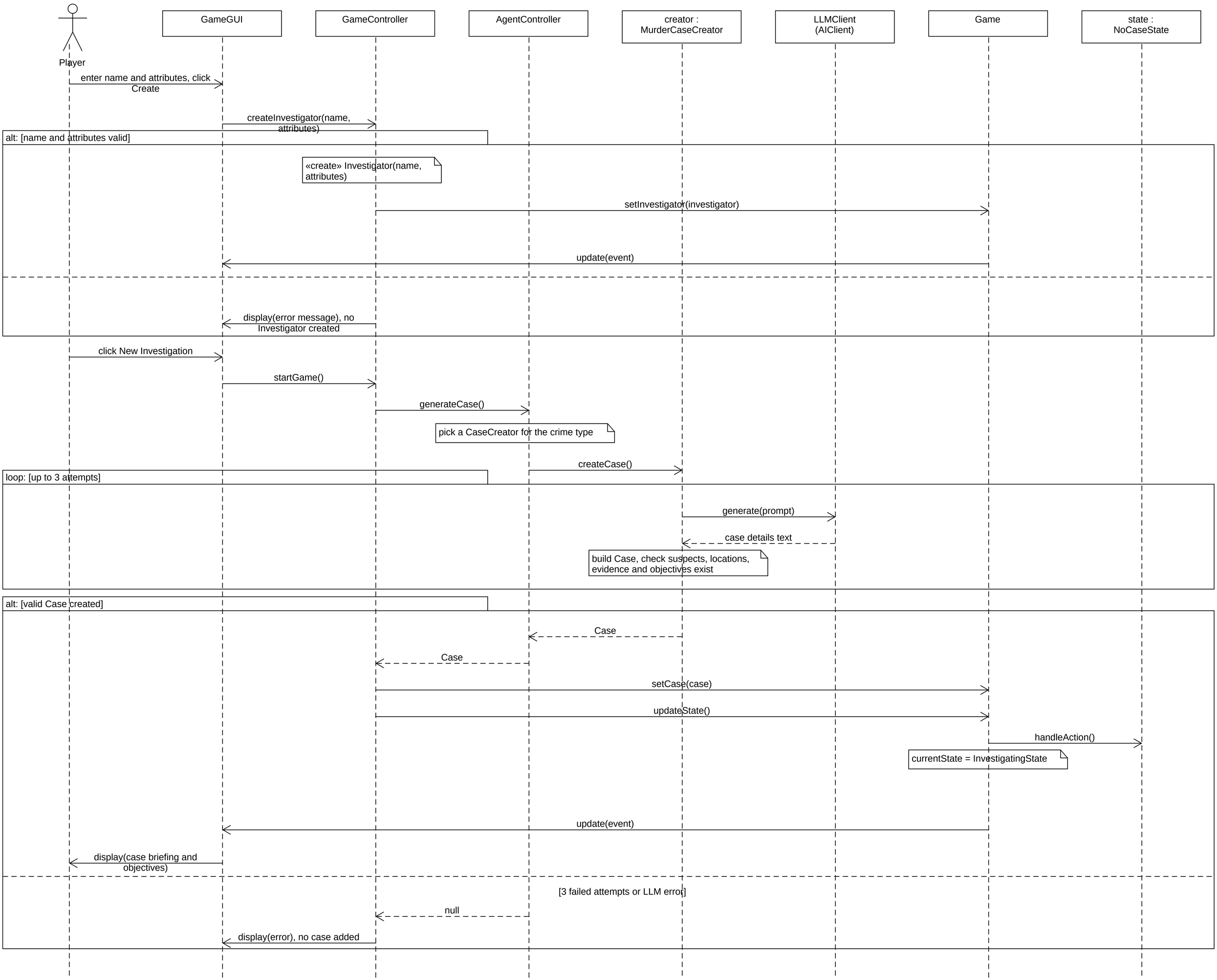
UC13 - Load Game

Actor(s)	Player, File System
Goal	Continue a saved investigation.
Preconditions	A save file exists.
Trigger	Player clicks Load (or uses the load command).
Main Success Scenario	1. Player clicks Load. 2. GameController.loadGame() calls Game.load(file). 3. Game reads the file and checks it is complete and valid. 4. Game restores its data and state and calls notifyObservers(gameLoaded). 5. All panels refresh.
Alternative / Exception Flows	3a. File missing or corrupted: rejected, player is told, the current game is unchanged.
Postconditions	Game matches the saved investigation.
Related Feature(s)	F12
Sequence Diagram	SD04

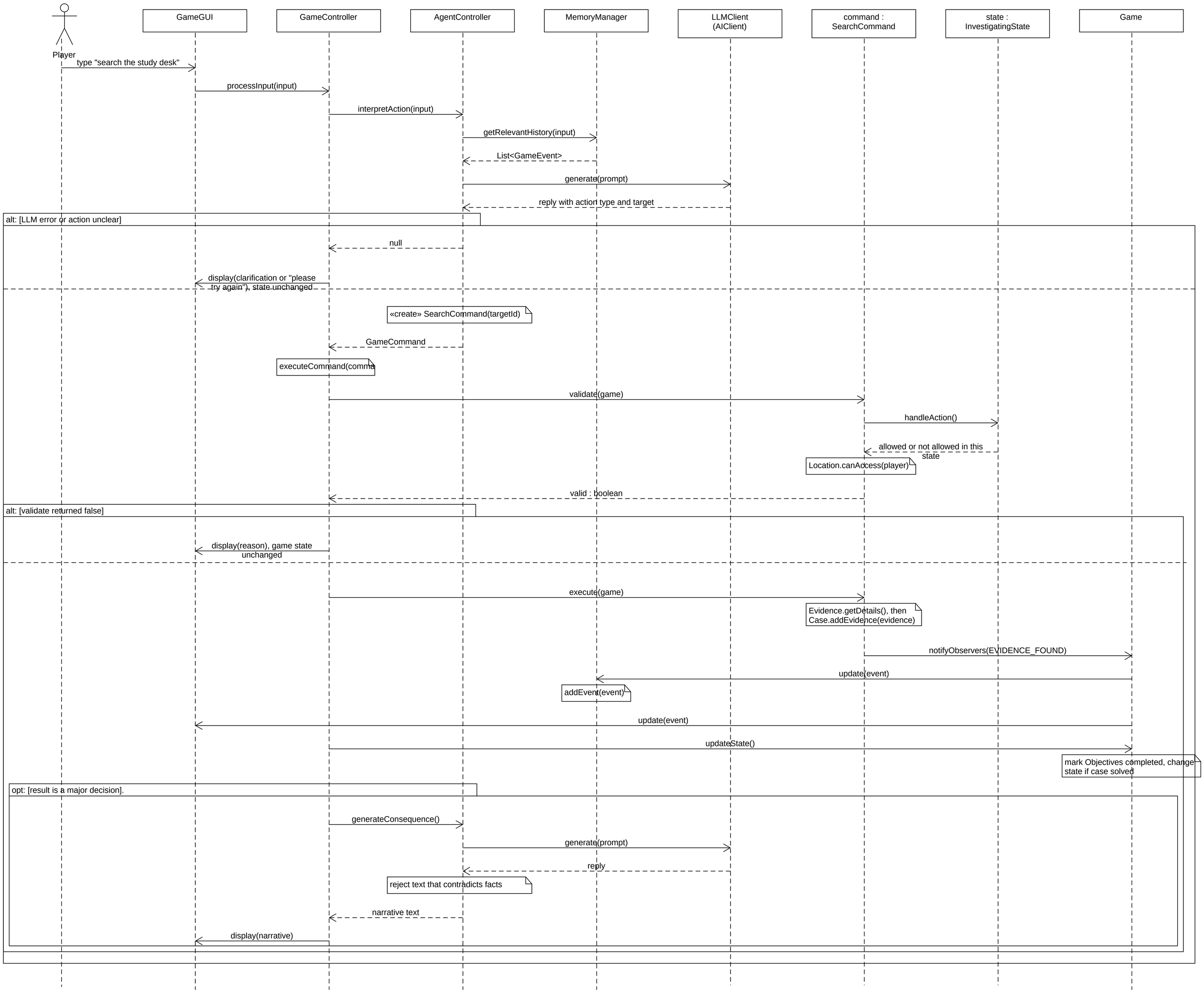
2.3 Sequence Diagrams

Five sequence diagrams cover the important behaviour of the system. Each one starts from the player and uses the classes and methods from the class diagram. Where several features follow the same steps they share one diagram.

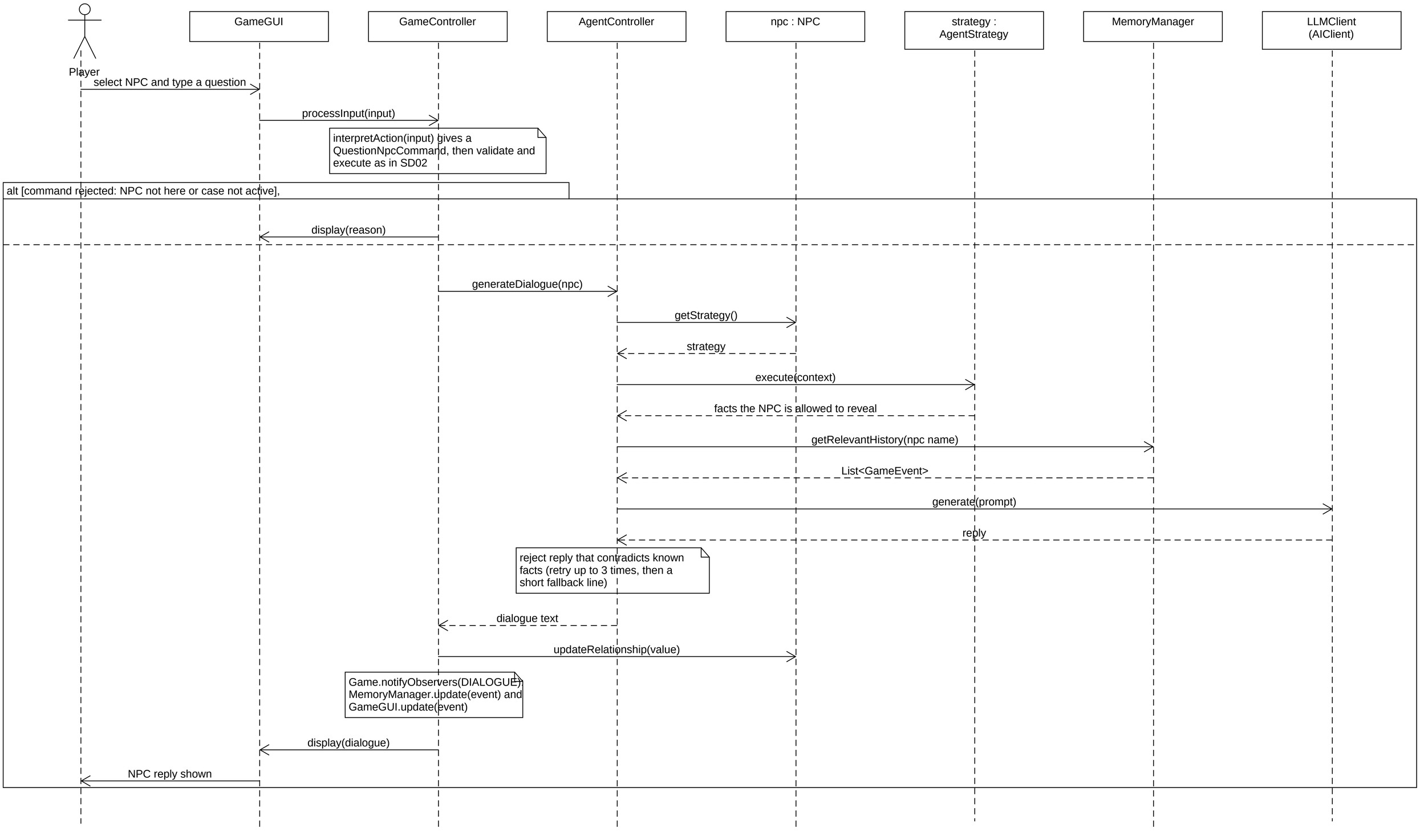
SD01: Create Investigator and Start New Investigation



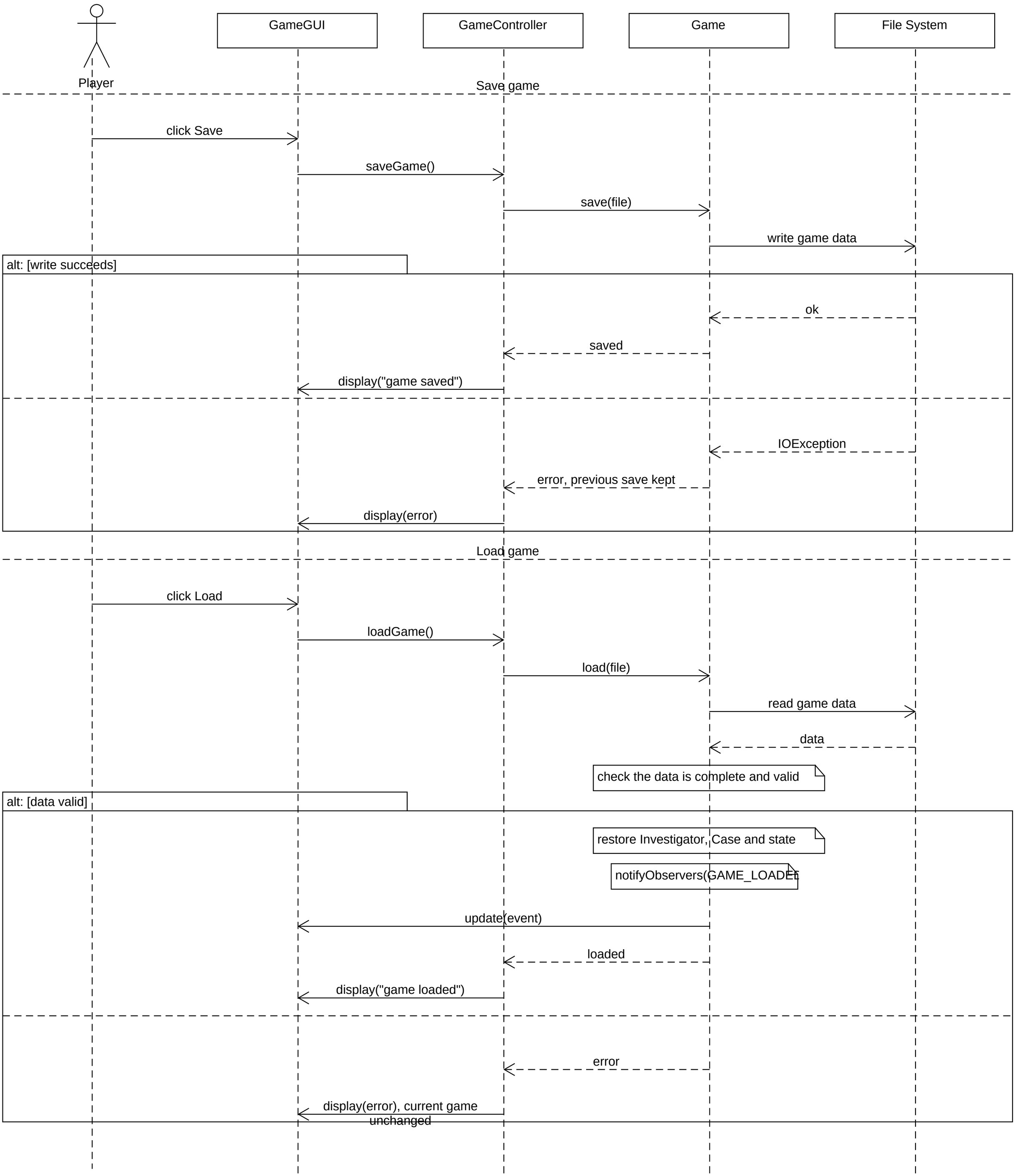
SD02: Perform a Natural Language Action



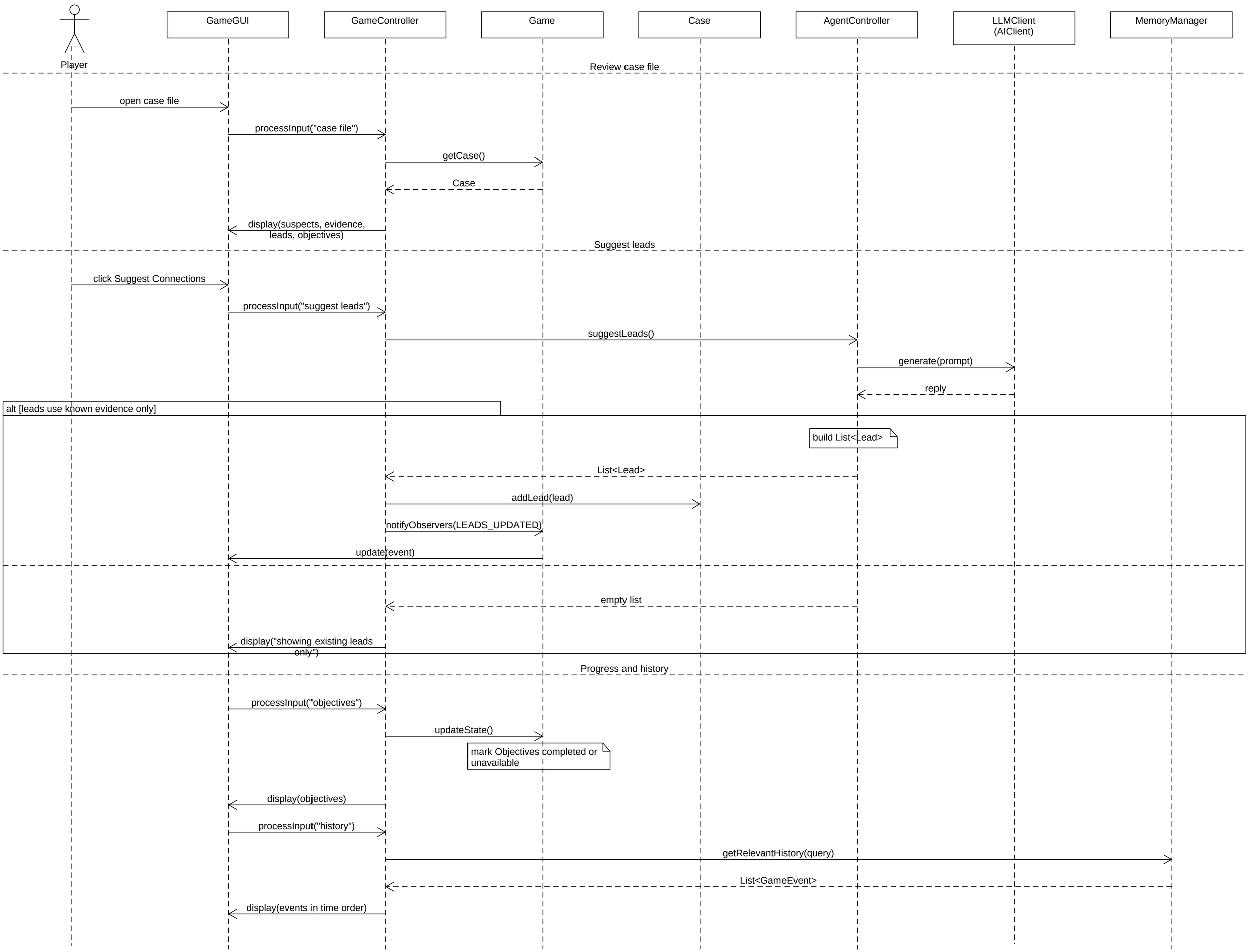
SD03: Interview NPC



SD04: Save and Load Game



SD05: Review Case File, Leads, Progress and History



SD01 - Create Investigator and Start New Investigation (F01, F02)

The player creates an investigator, then starts a new case. AgentController picks a CaseCreator, the creator asks LLMClient for the case details, and the case is checked before Game.setCase() is called. NoCaseState then moves the game to InvestigatingState. The diagram also shows the error cases for invalid input and for a case that cannot be generated.

SD02 - Perform a Natural Language Action (F03, F04, F06, F08, F09)

The player types an action and AgentController turns it into a GameCommand using MemoryManager and LLMClient. The command is validated (GameState and Location), executed, and the observers are notified. After a major decision, generateConsequence() adds a narrative. The diagram uses SearchCommand as the example. MoveCommand follows the same steps, and QuestionNpcCommand continues in SD03.

SD03 - Interview NPC (F05, F10)

A QuestionNpcCommand is accepted, then AgentController gets the NPC's AgentStrategy, uses it to decide which facts can be revealed, and asks LLMClient for the reply. Replies that contradict known facts are rejected and retried. The relationship is updated and the dialogue is stored through the observers.

SD04 - Save and Load Game (F12)

Save and load go from GameController to Game and the file system without the AI. The diagram shows a failed write and a missing or corrupted file, where the current game is kept.

SD05 - Review Case File, Leads, Progress and History (F07, F10, F11)

The player opens the case file, asks for suggested leads (AgentController.suggestLeads() with LLMClient), and looks at objectives and history. Suggestions that use unknown evidence are rejected, so only existing leads are shown.

3. Feature-to-Design Traceability

Table 1 links each feature to its use case, classes, key methods, sequence diagram and design patterns.

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01	Player creation	Deterministic	UC01 Create Investigator	GameGUI, GameController, Investigator, Game	createInvestigator(), setInvestigator(), getInfo()	SD01	Observer
F02	Dynamic case generation	Hybrid	UC02 Start New Investigation	GameController, AgentController, CaseCreator, MurderCaseCreator, TheftCaseCreator, LLMClient, Case, Game, NoCaseState	startGame(), generateCase(), createCase(), generate(), setCase(), updateState()	SD01	Factory Method, State
F03	World and location exploration	Hybrid	UC03 Explore Location	GameController, MoveCommand, GameState, Location, Game	executeCommand(), validate(), execute(), canAccess(), getDescription()	SD02	Command, State
F04	Natural language actions	Hybrid	UC04 Perform Natural Language Action	GameController, AgentController, MemoryManager, LLMClient, GameCommand	processInput(), interpretAction(), getRelevantHistory(), generate()	SD02	Command
F05	NPC interaction and dynamic dialogue	Hybrid	UC05 Interview NPC	QuestionNpcCommand, AgentController, NPC, AgentStrategy, TruthfulStrategy, EvasiveStrategy, MemoryManager, LLMClient	generateDialogue(), getStrategy(), execute(), getRelevantHistory(), updateRelationship()	SD03	Strategy, Command
F06	Evidence collection and management	Hybrid	UC06 Collect Evidence	SearchCommand, Evidence, Case, Game	validate(), execute(), getDetails(), addEvidence(), notifyObservers()	SD02	Command, Observer
F07	Investigation and lead management	Hybrid	UC07 Review Case File and Leads	GameController, AgentController, LLMClient, Case, Lead, Game	getCase(), suggestLeads(), addLead(), notifyObservers()	SD05	Observer
F08	Rule and action validation	Deterministic	UC08 Validate Action	GameController, GameCommand, GameState, InvestigatingState, Location	executeCommand(), validate(), handleAction(), canAccess()	SD02	Command, State
F09	Consequence generation	Hybrid	UC09 Apply Consequences	GameController, AgentController, LLMClient, Game, GameGUI	updateState(), generateConsequence(), generate(), display()	SD02	State, Observer

F10	Story and investigation memory	Hybrid	UC05, UC09, UC10 Review Investigation History	MemoryManager, GameEvent, GameObserver, Game, AgentController	update(), addEvent(), getRelevantHistory(), notifyObservers()	SD02, SD03, SD05	Observer
F11	Case progress tracking	Hybrid	UC11 Track Case Progress	Game, Case, Objective, GameState, SolvedState, GameGUI	updateState(), updateStatus(), getStateName()	SD05, SD02	State
F12	Save and load game	Deterministic	UC12 Save Game, UC13 Load Game	GameController, Game, GameGUI	saveGame(), loadGame(), save(), load(), notifyObservers()	SD04	Observer

Feature-to-Design Traceability

4. Feature Implementation Explanations

F01 - Player Creation

Related Use Case: UC01 - Create Investigator

Related Sequence Diagram: SD01

Classes involved: GameGUI (collects the name and attributes and shows the profile), GameController (checks the input and creates the investigator), Investigator (stores name, attributes and reputation), Game (stores the investigator and notifies observers).

Important methods: GameController.createInvestigator(), Game.setInvestigator(), Game.notifyObservers(), GameGUI.update(), Investigator.getInfo().

Execution: When the player clicks Create, GameGUI calls createInvestigator() with the name and attributes. GameController checks that the name is not empty and the attributes are valid, then creates an Investigator and gives it to Game with setInvestigator(). Game sends a playerCreated event to its observers and GameGUI shows the profile from getInfo(). If the input is invalid, an error is displayed and no Investigator is created.

F02 - Dynamic Case Generation

Related Use Case: UC02 - Start New Investigation

Related Sequence Diagram: SD01

Classes involved: GameGUI (New Investigation button), GameController (starts the process), AgentController (picks a creator), CaseCreator with MurderCaseCreator and TheftCaseCreator (build the Case), LLMClient and AIClient (talk to the LLM), Case, Game, and NoCaseState and InvestigatingState.

Important methods: GameController.startGame(), AgentController.generateCase(), CaseCreator.createCase(), LLMClient.generate(), Game.setCase(), Game.updateState(), GameState.handleAction().

Execution: GameController.startGame() calls generateCase(). AgentController picks a CaseCreator and calls createCase(), and the creator asks LLMClient.generate() for the case

details. The creator checks that suspects, locations, evidence and objectives exist, and tries again up to 3 times if not. GameController then calls setCase() and updateState(), and NoCaseState moves the game to InvestigatingState. If all attempts fail, an error is shown and no case is added.

F03 - World and Location Exploration

Related Use Case: UC03 - Explore Location

Related Sequence Diagram: SD02

Classes involved: GameGUI (map), GameController, MoveCommand, GameState, Location and Game.

Important methods: GameController.executeCommand(), MoveCommand.validate(), MoveCommand.execute(), Location.canAccess(), Location.getDescription().

Execution: Selecting a location creates a MoveCommand that is sent to executeCommand(). validate() asks the current GameState and Location.canAccess(player). If it is valid, execute() changes the current location, Game notifies its observers, and GameGUI shows getDescription(). If the location is restricted, the reason is shown and nothing changes. SD02 draws SearchCommand as its example, and MoveCommand follows the same steps.

F04 - Natural Language Actions

Related Use Case: UC04 - Perform Natural Language Action

Related Sequence Diagram: SD02

Classes involved: GameGUI and GameCLI (input), GameController, AgentController (interprets the text), MemoryManager (relevant history), LLMClient, GameCommand.

Important methods: GameController.processInput(), AgentController.interpretAction(), MemoryManager.getRelevantHistory(), LLMClient.generate().

Execution: processInput() passes the text to interpretAction(). The agent gets relevant events from MemoryManager, asks LLMClient.generate() what the action and target are, and creates the matching GameCommand. If the LLM fails or the request is unclear, null is

returned and GameController asks the player to try again or clarify without changing the state. A valid command goes on to executeCommand().

F05 - NPC Interaction and Dynamic Dialogue

Related Use Case: UC05 - Interview NPC

Related Sequence Diagram: SD03

Classes involved: QuestionNpcCommand (the validated action), AgentController, NPC, AgentStrategy with TruthfulStrategy and EvasiveStrategy, MemoryManager, LLMClient.

Important methods: AgentController.generateDialogue(), NPC.getStrategy(), AgentStrategy.execute(), MemoryManager.getRelevantHistory(), LLMClient.generate(), NPC.updateRelationship().

Execution: After the QuestionNpcCommand is accepted, generateDialogue(npc) gets the NPC's strategy with getStrategy(). The strategy's execute() decides which facts the NPC may reveal. The agent adds relevant history and asks LLMClient for the reply, rejecting any reply that contradicts known facts (up to 3 tries, then a short fallback line). GameController then calls updateRelationship() and the reply is displayed.

F06 - Evidence Collection and Management

Related Use Case: UC06 - Collect Evidence

Related Sequence Diagram: SD02

Classes involved: SearchCommand, Evidence, Case, Game, GameGUI.

Important methods: SearchCommand.validate(), SearchCommand.execute(), Evidence.getDetails(), Case.addEvidence(), Game.notifyObservers().

Execution: A search action becomes a SearchCommand. After validation, execute() finds the Evidence for the target and calls Case.addEvidence(). Game then notifies the observers so the case file in GameGUI updates. If there is nothing to find or the evidence was already found, the player is told and nothing is added.

F07 - Investigation and Lead Management

Related Use Case: UC07 - Review Case File and Leads

Related Sequence Diagram: SD05

Classes involved: GameController, Game, Case, Lead, AgentController, LLMClient, GameGUI.

Important methods: Game.getCase(), AgentController.suggestLeads(), LLMClient.generate(), Case.addLead(), Game.notifyObservers().

Execution: Opening the case file uses Game.getCase() and GameGUI displays it. When the player asks for suggestions, suggestLeads() asks LLMClient for possible leads and keeps only the ones that use known evidence. GameController adds each one with Case.addLead() and Game notifies the observers so the lead board refreshes. If the suggestion is invalid or the LLM fails, only the existing leads are shown.

F08 - Rule and Action Validation

Related Use Case: UC08 - Validate Action

Related Sequence Diagram: SD02

Classes involved: GameController, GameCommand and its concrete commands, GameState with InvestigatingState, Location.

Important methods: GameController.executeCommand(), GameCommand.validate(), GameState.handleAction(), Location.canAccess().

Execution: executeCommand() always calls validate(game) first. The command asks the current GameState through handleAction() whether the action is allowed in this phase, then checks its own rules, for example Location.canAccess(). Only if validate() returns true does GameController call execute(). If it returns false, the reason is displayed and the state does not change.

F09 - Consequence Generation

Related Use Case: UC09 - Apply Consequences

Related Sequence Diagram: SD02

Classes involved: GameController, Game, AgentController, LLMClient, GameGUI.

Important methods: Game.updateState(), AgentController.generateConsequence(), LLMClient.generate(), GameGUI.display().

Execution: After a command runs, updateState() applies the deterministic changes first. For a major decision GameController then calls generateConsequence(), which asks LLMClient for a narrative and rejects text that contradicts the known facts. The narrative is sent to GameGUI.display(). If the LLM is unavailable, a plain text description of the changes is shown instead.

F10 - Story and Investigation Memory

Related Use Case: UC10 - Review Investigation History

Related Sequence Diagram: SD02, SD03, SD05

Classes involved: MemoryManager, GameEvent, GameObserver, Game, AgentController, GameGUI.

Important methods: Game.notifyObservers(), MemoryManager.update(), MemoryManager.addEvent(), MemoryManager.getRelevantHistory().

Execution: MemoryManager is an observer of Game. Each time Game calls notifyObservers(), MemoryManager.update() receives the GameEvent and stores it with addEvent(). Before asking the LLM for anything, AgentController calls getRelevantHistory(query), so only the relevant events are in the prompt. For the history view, GameController calls the same method and GameGUI shows the events in time order.

F11 - Case Progress Tracking

Related Use Case: UC11 - Track Case Progress

Related Sequence Diagram: SD05, SD02

Classes involved: Game, Case, Objective, GameState with SolvedState, GameGUI.

Important methods: Game.updateState(), Case.updateStatus(), GameState.getStateName().

Execution: After each valid action GameController calls Game.updateState(). Game compares the current state with each Objective in the Case, marks the completed ones, and marks an objective unavailable if a decision made it impossible. When the case is solved, the state changes to SolvedState and GameGUI shows the completed and active objectives.

F12 - Save and Load Game

Related Use Case: UC12 - Save Game and UC13 - Load Game

Related Sequence Diagram: SD04

Classes involved: GameGUI, GameController, Game and the file system.

Important methods: GameController.saveGame(), GameController.loadGame(), Game.save(file), Game.load(file), Game.notifyObservers().

Execution: Save calls saveGame(), which calls Game.save(file) to write the player, case, evidence, relationships, progress and history. If the write fails, an error is shown and the previous save is kept. Load calls loadGame() and Game.load(file), which reads the file and checks if it is complete and valid. If valid, Game restores its data and state and sends a gameLoaded event so all panels refresh. A missing or corrupted file is rejected and the current game is unchanged.